

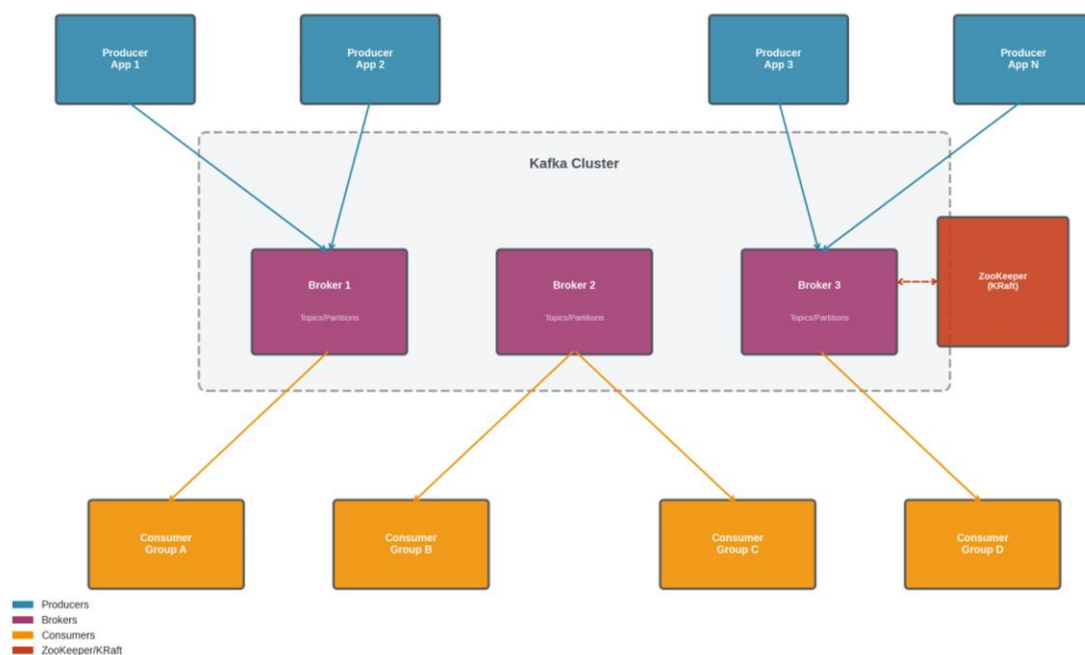
FATEC VOTORANTIM

Ciência de Dados em Negócios

5º Semestre de Ciência de Dados em Negócios

Introdução à solução e seus benefícios, objetivos do projeto, descrição da plataforma Apache Kafka, BPV (Business Process View) aplicado à automação orientada a eventos com diagrama, e fluxograma lógico de instalação e configuração do ambiente

Arquitetura do Apache Kafka



EQUIPE DE FÁBRICA DE SOFTWARE:

Equipe: Arthur Nunes, Bruno Araujo e Caio César Corr 

Sorocaba/SP

2026

Introdução ao Estudo da Solução

O crescimento exponencial do volume de dados gerados por aplicações modernas exige plataformas capazes de realizar ingestão, distribuição e processamento de eventos em tempo real com alta confiabilidade e escalabilidade.

Nesse contexto, o Apache Kafka surge como uma plataforma distribuída de streaming projetada para lidar com grandes volumes de dados em tempo real, permitindo a construção de arquiteturas orientadas a eventos (Event-Driven Architecture).

Diferente de sistemas tradicionais de mensageria, o Kafka funciona como um commit log distribuído, garantindo:

- Alta taxa de throughput
- Baixa latência
- Persistência e reprocessamento de eventos
- Escalabilidade horizontal
- Tolerância a falhas

Esta Prova de Conceito (POC) tem como objetivo apresentar a instalação, configuração e validação de um ambiente Kafka em Linux, demonstrando sua aplicação em cenários de processamento de dados em larga escala.

Objetivo do Projeto

O projeto tem como objetivo implementar um ambiente de streaming de dados utilizando o Apache Kafka, possibilitando a ingestão, armazenamento e distribuição de eventos em tempo real de forma eficiente, escalável e automatizada.

A solução busca demonstrar como plataformas modernas de mensageria distribuída podem ser configuradas e utilizadas para manipular grandes volumes de dados em fluxo contínuo, explorando conceitos como baixa latência, arquitetura orientada a eventos (event-driven), tolerância a falhas, alta disponibilidade e escalabilidade horizontal.

A seguir, são apresentados os principais objetivos do projeto, organizados conforme as etapas de implantação, configuração e operação da plataforma.

- **Preparar ambiente Linux para execução do Kafka**

Realizar a preparação do sistema operacional Linux com as configurações básicas de rede, usuários, permissões e estrutura de diretórios necessárias para suportar a execução de serviços distribuídos. Esta etapa garante estabilidade, segurança e organização do ambiente que hospedará o broker Kafka.

- **Instalar e configurar Java e dependências**

Efetuar a instalação do OpenJDK e a configuração das variáveis de ambiente necessárias para execução de aplicações baseadas na JVM. Esta etapa assegura compatibilidade com o Kafka e prepara o ambiente para execução confiável dos serviços.

- **Instalar e configurar Apache Kafka**

Realizar o download, extração e configuração inicial do Apache Kafka, incluindo parâmetros de broker, armazenamento de logs, portas de comunicação e integração com o ZooKeeper (modo legado). O objetivo é disponibilizar um broker funcional e pronto para receber eventos.

- **Validar comunicação Producer → Broker → Consumer**

Executar testes práticos de mensageria utilizando ferramentas nativas do Kafka para criação de tópicos, envio de mensagens (producer) e consumo de eventos (consumer), comprovando o funcionamento completo do fluxo de dados.

- **Demonstrar arquitetura de streaming orientada a eventos**

Evidenciar o funcionamento da arquitetura publish/subscribe, mostrando como aplicações produtoras e consumidoras podem ser desacopladas e escaláveis, permitindo o processamento assíncrono e distribuído de eventos.

- **Documentar arquitetura e fluxos operacionais**

Registrar toda a arquitetura implementada, fluxos de dados, etapas de instalação e testes realizados, garantindo rastreabilidade e servindo como base para futuras evoluções e implantação em ambientes maiores.

VISÃO GERAL DO APACHE KAFKA

O Apache Kafka é uma plataforma distribuída baseada no modelo publish/subscribe, onde produtores enviam mensagens para tópicos e consumidores leem esses eventos de forma assíncrona.

Componentes Principais

Componente	Descrição
Producer	Publica eventos nos tópicos
Broker	Servidor Kafka que armazena e distribui mensagens
Topic	Canal lógico de eventos
Partition	Subdivisão de um tópico para paralelismo
Consumer	Aplicação que consome eventos
Consumer Group	Grupo de consumidores que divide carga
ZooKeeper (modo legado)	Coordenação de cluster

Producer

O Producer é a aplicação responsável por publicar eventos no cluster Kafka. Do ponto de vista de infraestrutura, o foco está na conectividade e na configuração correta dos parâmetros de envio.

Aspectos de instalação e infraestrutura

- Necessita apenas do cliente Kafka (CLI ou biblioteca).
- Deve conhecer o endereço do broker (bootstrap.servers).
- Pode rodar em qualquer servidor ou aplicação externa.
- Requer acesso à porta **9092/TCP** do broker.
- Pode ser configurado para compressão, retries e confirmação de entrega (acks).

Responsabilidade na arquitetura

- Enviar eventos de forma confiável.
- Definir chave de particionamento (key) quando necessário.
- Garantir alta taxa de ingestão.

Broker

O Broker é o servidor central do Kafka responsável por armazenar, replicar e disponibilizar mensagens.

Aspectos de instalação e infraestrutura

- Instalação do binário Kafka no Linux.
- Configuração do arquivo server.properties.
- Definição de:
 - broker.id
 - listeners (porta 9092)
 - log.dirs (diretório de armazenamento)
 - conexão com ZooKeeper.
- Necessita de:
 - CPU e RAM dedicadas
 - Disco rápido (preferencialmente SSD)
 - Porta **9092** liberada.

Responsabilidade na arquitetura

- Persistência de eventos em disco.
- Replicação e tolerância a falhas.
- Balanceamento de carga entre consumidores.

Topic

O Topic é a entidade lógica onde os eventos são armazenados.

Aspectos de instalação e infraestrutura

- Criado via CLI do Kafka.
- Pode ser configurado com:
 - número de partições
 - fator de replicação
 - política de retenção de dados
- Armazenado fisicamente no diretório log.dirs.

Responsabilidade na arquitetura

- Organizar eventos por domínio de negócio.
- Permitir retenção e reprocessamento de dados.

Partition

Partições são subdivisões de um tópico que permitem paralelismo e escalabilidade.

Aspectos de instalação e infraestrutura

- Definidas na criação do tópico.
- Distribuídas automaticamente entre brokers.
- Influenciam diretamente a capacidade de throughput.

Responsabilidade na arquitetura

- Permitir processamento paralelo.
- Garantir ordenação dentro de cada partição.
- Escalar horizontalmente o cluster.

Consumer

O Consumer é a aplicação responsável por ler eventos dos tópicos.

Aspectos de instalação e infraestrutura

- Utiliza cliente Kafka (CLI ou biblioteca).
- Conecta ao broker via bootstrap.servers.
- Requer acesso à porta **9092/TCP**.
- Pode ser executado em múltiplos servidores.

Responsabilidade na arquitetura

- Processar eventos no próprio ritmo.
- Manter controle de offset de leitura.
- Permitir reprocessamento de dados.

Consumer Group

Conjunto de consumidores que compartilham a leitura de um tópico.

Aspectos de instalação e infraestrutura

- Configurado via parâmetro group.id.
- Kafka distribui partições automaticamente entre os membros.
- Requer coordenação via ZooKeeper (modo legado).

Responsabilidade na arquitetura

- Escalabilidade horizontal do consumo.
- Balanceamento automático de carga.
- Alta disponibilidade (failover automático).

ZooKeeper (modo legado)

Serviço responsável pela coordenação do cluster Kafka no modo tradicional.

Aspectos de instalação e infraestrutura

- Instalação como serviço Linux.
- Porta padrão **2181/TCP**.
- Armazena metadados do cluster:
 - brokers ativos
 - líderes de partição
 - configuração de tópicos
- Deve iniciar antes do Kafka.

Responsabilidade na arquitetura

- Eleição de líder.
- Coordenação do cluster.
- Gerenciamento de configuração distribuída.

BPV (Business Process View) automação de negócios:

Visão Geral:

A empresa atua na distribuição e locação de chopeiras, realizando o monitoramento contínuo do consumo de chopp em eventos e estabelecimentos comerciais. Cada equipamento possui sensores embarcados responsáveis por coletar informações em tempo real, como volume consumido, temperatura da bebida e status operacional da chopeira.

Para garantir escalabilidade, confiabilidade e processamento em tempo real, a solução adota uma arquitetura orientada a eventos baseada em Apache Kafka como backbone de mensageria. Os dados gerados pelos sensores são publicados continuamente em tópicos Kafka, permitindo que múltiplos sistemas consumam essas informações de forma desacoplada e assíncrona.

O processamento analítico dos dados é realizado por aplicações de streaming (como Apache Flink), que consomem os eventos do Kafka para identificar padrões de consumo, prever reposição de barris, detectar falhas operacionais e acionar processos automáticos. Essa arquitetura possibilita tomada de decisão em tempo real, maior eficiência logística e redução de desperdícios, além de permitir a evolução futura da plataforma com novos serviços consumidores sem impacto nos produtores de dados.

Descrição do Negócio:

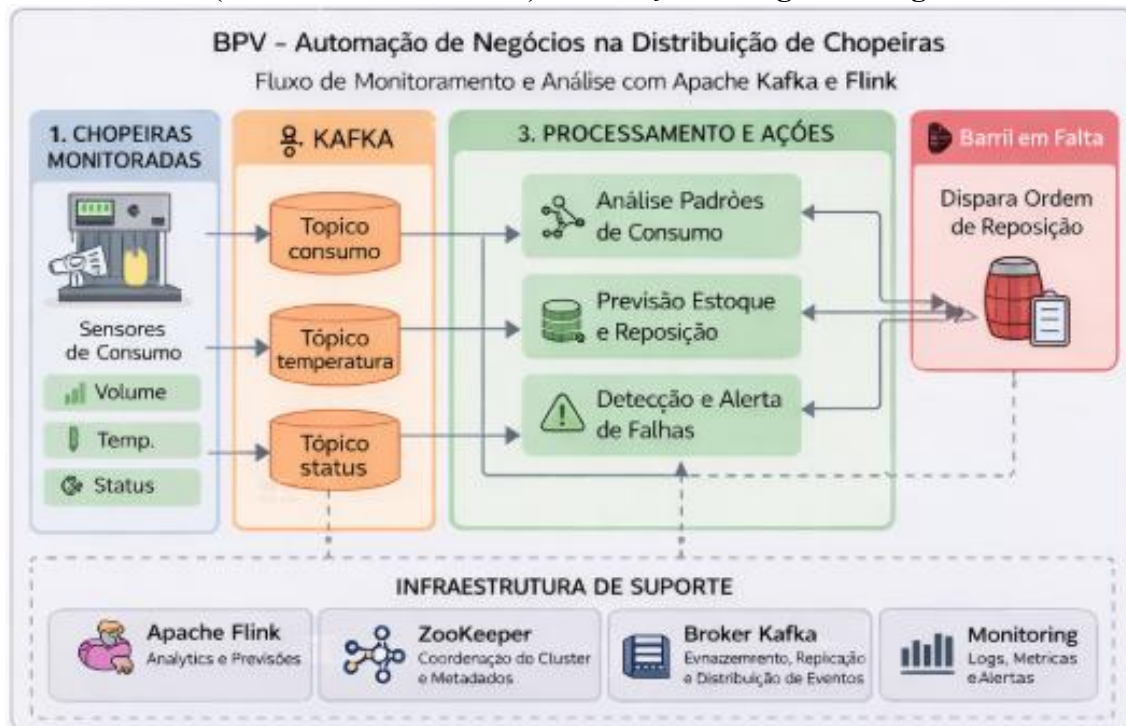
O fluxo de automação inicia-se nas chopeiras monitoradas, onde sensores IoT capturam continuamente dados de telemetria relacionados ao consumo de bebida, temperatura e estado do equipamento. Esses dados são enviados para a plataforma e publicados em tópicos do Apache Kafka, organizados por domínio de informação, como consumo, temperatura e status.

O Kafka atua como camada central de ingestão e distribuição de eventos, garantindo persistência, ordenação e disponibilidade dos dados. A partir desse ponto, diferentes serviços consumidores podem acessar os eventos de forma paralela, sem interferir entre si.

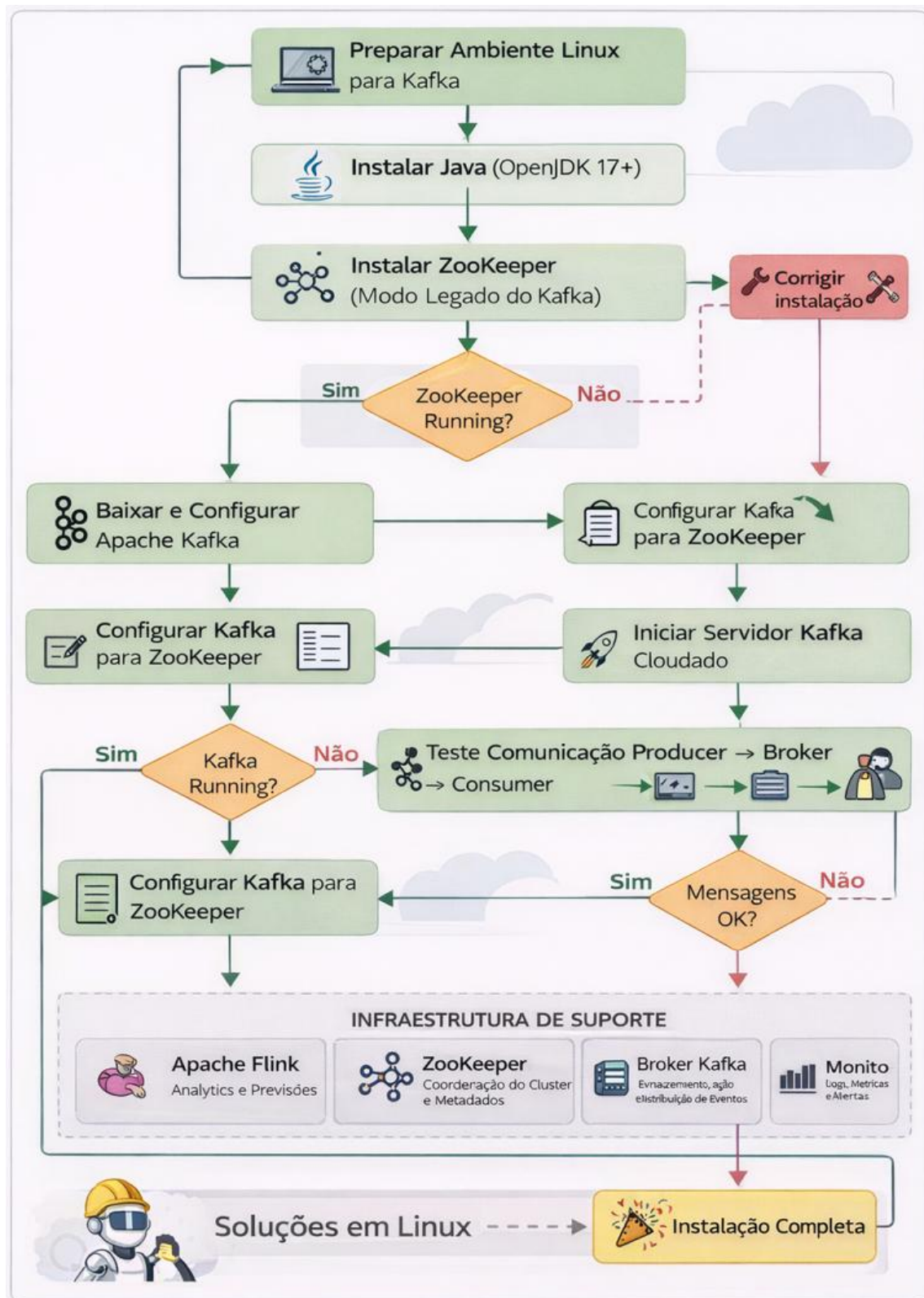
Entre os consumidores, destacam-se as aplicações de processamento em tempo real responsáveis por analisar o comportamento de consumo. Esses serviços processam os fluxos de dados para identificar padrões, calcular previsões de demanda e detectar possíveis anomalias operacionais. Quando o sistema identifica que um barril está próximo do esgotamento ou detecta falhas no equipamento, eventos derivados são gerados e enviados para sistemas logísticos e operacionais.

Como resultado, ordens automáticas de reposição podem ser disparadas, equipes de suporte podem ser notificadas e dashboards analíticos são atualizados em tempo real. Toda a operação é monitorada por mecanismos de logs, métricas e alertas, garantindo observabilidade e alta disponibilidade do ambiente.

BPV (Business Process View) automação de negócios diagrama



▪ Fluxograma Lógico da Solução Instalação e Configuração



- Evidências da implementação realizada em VM Linux / Ubuntu

```
cdn@cdn-Virtual-Machine: /opt/kafka
Executing: /lib/systemd/systemd-sysv-install enable zookeeper
cdn@cdn-Virtual-Machine:/$ sudo systemctl start zookeeper
cdn@cdn-Virtual-Machine:/$ netstat -tulnp | grep 2181
Command 'netstat' not found, but can be installed with:
sudo apt install net-tools
cdn@cdn-Virtual-Machine:/$ ^C
cdn@cdn-Virtual-Machine:/$ netstat -tulnp | grep 2181
Command 'netstat' not found, but can be installed with:
sudo apt install net-tools
cdn@cdn-Virtual-Machine:/$ sudo apt install net-tools
Lendo listas de pacotes... Pronto
Construindo árvore de dependências... Pronto
Lendo informação de estado... Pronto
Os NOVOS pacotes a seguir serão instalados:
  net-tools
0 pacotes atualizados, 1 pacotes novos instalados, 0 a serem removidos e 2 não atualizados.
É preciso baixar 204 kB de arquivos.
Depois desta operação, 819 kB adicionais de espaço em disco serão usados.
Obter:1 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 net-tools amd64 1.60+git2018110
3.0eebece-1ubuntu5.4 [204 kB]
Baixados 204 kB em 1s (148 kB/s)
A seleccionar pacote anteriormente não seleccionado net-tools.
(Lendo banco de dados ... 221571 ficheiros e directórios actualmente instalados.)
A preparar para desempacotar .../net-tools_1.60+git20181103.0eebece-1ubuntu5.4_amd64.deb ...
A descompactar net-tools (1.60+git20181103.0eebece-1ubuntu5.4) ...
Configurando net-tools (1.60+git20181103.0eebece-1ubuntu5.4) ...
A processar 'triggers' para man-db (2.10.2-1) ...
cdn@cdn-Virtual-Machine:/$ netstat -tulnp | grep 2181
(Not all processes could be identified, non-owned process info
 will not be shown, you would have to be root to see it all.)
tcp6      0      0 :::2181                :::*                    LISTEN     -

cdn@cdn-Virtual-Machine:/$ echo ruok | nc localhost 2181
imokcdn@cdn-Virtual-Machine:/$
cdn@cdn-Virtual-Machine:/$ cd /opt
cdn@cdn-Virtual-Machine:/opt$ sudo wget https://downloads.apache.org/kafka/3.7.0/kafka_2.13-3.7.0
.tgz
--2026-04-12 16:58:22-- https://downloads.apache.org/kafka/3.7.0/kafka_2.13-3.7.0.tgz
Resolving downloads.apache.org (downloads.apache.org)... 88.99.208.237, 135.181.214.104, 2a01:4f8
:10a:39da::2, ...
Connecting to downloads.apache.org (downloads.apache.org)|88.99.208.237|:443... connected.
HTTP request sent, awaiting response... 404 Not Found
2026-04-12 16:58:23 ERROR 404: Not Found.

cdn@cdn-Virtual-Machine:/opt$ sudo wget https://downloads.apache.org/kafka/3.6.1/kafka_2.13-3.6.1
.tgz
--2026-04-12 16:59:03-- https://downloads.apache.org/kafka/3.6.1/kafka_2.13-3.6.1.tgz
Resolving downloads.apache.org (downloads.apache.org)... 135.181.214.104, 88.99.208.237, 2a01:4f8
:10a:39da::2, ...
Connecting to downloads.apache.org (downloads.apache.org)|135.181.214.104|:443... connected.
HTTP request sent, awaiting response... 404 Not Found
2026-04-12 16:59:04 ERROR 404: Not Found.

cdn@cdn-Virtual-Machine:/opt$ sudo wget https://downloads.apache.org/kafka/3.9.2/kafka_2.13-3.9.2
.tgz
--2026-04-12 17:00:41-- https://downloads.apache.org/kafka/3.9.2/kafka_2.13-3.9.2.tgz
Resolving downloads.apache.org (downloads.apache.org)... 135.181.214.104, 88.99.208.237, 2a01:4f9
:3a:2c57::2, ...
Connecting to downloads.apache.org (downloads.apache.org)|135.181.214.104|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 122473776 (117M) [application/x-gzip]
Saving to: 'kafka_2.13-3.9.2.tgz'

kafka_2.13-3.9.2.tgz      100%[=====] 116,80M  13,1MB/s   in 10s

2026-04-12 17:00:52 (11,3 MB/s) - 'kafka_2.13-3.9.2.tgz' saved [122473776/122473776]

cdn@cdn-Virtual-Machine:/opt$ sudo tar -xzf kafka_2.13-3.9.2.tgz
cdn@cdn-Virtual-Machine:/opt$ sudo mv kafka_2.13-3.9.2 kafka
cdn@cdn-Virtual-Machine:/opt$ sudo chown -R $USER:$USER /opt/kafka
cdn@cdn-Virtual-Machine:/opt$ cd /opt/kafka
cdn@cdn-Virtual-Machine:/opt/kafka$ nano config/server.properties
cdn@cdn-Virtual-Machine:/opt/kafka$ /opt/kafka/bin/kafka-server-start.sh config/server.properties
[2026-04-12 17:03:17,523] INFO Registered kafka:type=kafka.Log4jController MBean (kafka.utils.Log
```

```
cdn@cdn-Virtual-Machine: /opt/kafka

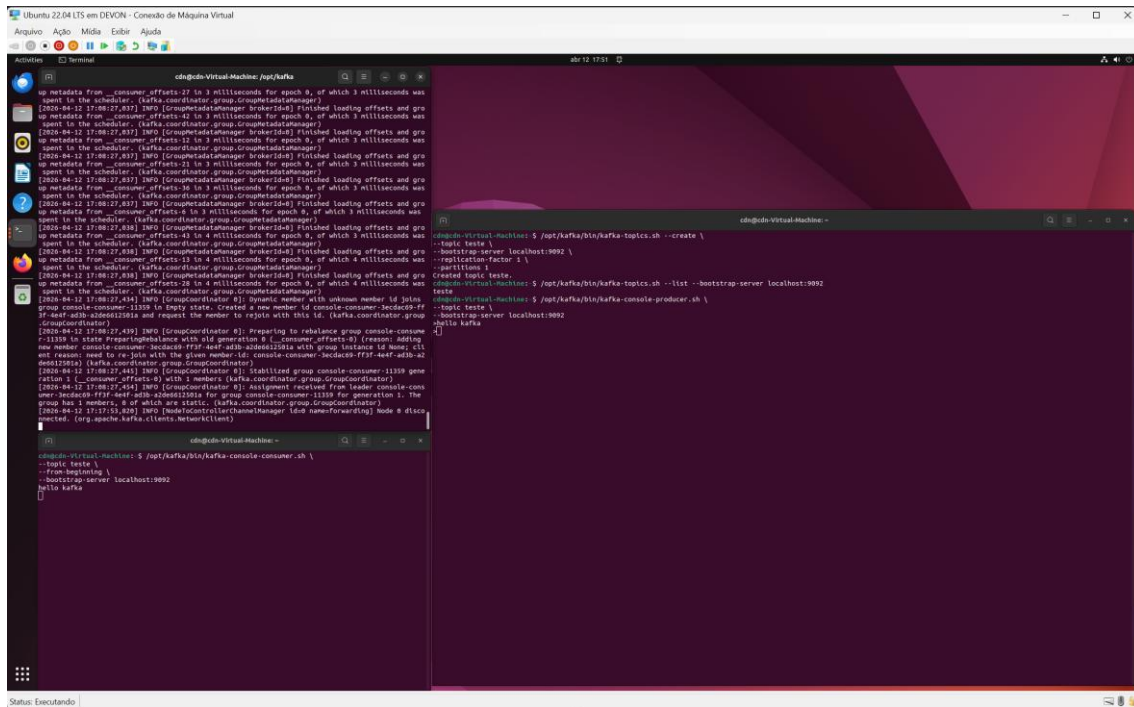
up metadata from __consumer_offsets-27 in 3 milliseconds for epoch 0, of which 3 milliseconds was
spent in the scheduler. (kafka.coordinator.group.GroupMetadataManager)
[2026-04-12 17:08:27,037] INFO [GroupMetadataManager brokerId=0] Finished loading offsets and gro
up metadata from __consumer_offsets-42 in 3 milliseconds for epoch 0, of which 3 milliseconds was
spent in the scheduler. (kafka.coordinator.group.GroupMetadataManager)
[2026-04-12 17:08:27,037] INFO [GroupMetadataManager brokerId=0] Finished loading offsets and gro
up metadata from __consumer_offsets-12 in 3 milliseconds for epoch 0, of which 3 milliseconds was
spent in the scheduler. (kafka.coordinator.group.GroupMetadataManager)
[2026-04-12 17:08:27,037] INFO [GroupMetadataManager brokerId=0] Finished loading offsets and gro
up metadata from __consumer_offsets-21 in 3 milliseconds for epoch 0, of which 3 milliseconds was
spent in the scheduler. (kafka.coordinator.group.GroupMetadataManager)
[2026-04-12 17:08:27,037] INFO [GroupMetadataManager brokerId=0] Finished loading offsets and gro
up metadata from __consumer_offsets-36 in 3 milliseconds for epoch 0, of which 3 milliseconds was
spent in the scheduler. (kafka.coordinator.group.GroupMetadataManager)
[2026-04-12 17:08:27,037] INFO [GroupMetadataManager brokerId=0] Finished loading offsets and gro
up metadata from __consumer_offsets-6 in 3 milliseconds for epoch 0, of which 3 milliseconds was
spent in the scheduler. (kafka.coordinator.group.GroupMetadataManager)
[2026-04-12 17:08:27,038] INFO [GroupMetadataManager brokerId=0] Finished loading offsets and gro
up metadata from __consumer_offsets-43 in 4 milliseconds for epoch 0, of which 3 milliseconds was
spent in the scheduler. (kafka.coordinator.group.GroupMetadataManager)
[2026-04-12 17:08:27,038] INFO [GroupMetadataManager brokerId=0] Finished loading offsets and gro
up metadata from __consumer_offsets-13 in 4 milliseconds for epoch 0, of which 4 milliseconds was
spent in the scheduler. (kafka.coordinator.group.GroupMetadataManager)
[2026-04-12 17:08:27,038] INFO [GroupMetadataManager brokerId=0] Finished loading offsets and gro
up metadata from __consumer_offsets-28 in 4 milliseconds for epoch 0, of which 4 milliseconds was
spent in the scheduler. (kafka.coordinator.group.GroupMetadataManager)
[2026-04-12 17:08:27,434] INFO [GroupCoordinator 0]: Dynamic member with unknown member id joins
group console-consumer-11359 in Empty state. Created a new member id console-consumer-3ecdac69-ff
3f-4e4f-ad3b-a2de6612501a and request the member to rejoin with this id. (kafka.coordinator.group
.GroupCoordinator)
[2026-04-12 17:08:27,439] INFO [GroupCoordinator 0]: Preparing to rebalance group console-consume
r-11359 in state PreparingRebalance with old generation 0 (__consumer_offsets-0) (reason: Adding
new member console-consumer-3ecdac69-ff3f-4e4f-ad3b-a2de6612501a with group instance id None; cli
ent reason: need to re-join with the given member-id: console-consumer-3ecdac69-ff3f-4e4f-ad3b-a2
de6612501a) (kafka.coordinator.group.GroupCoordinator)
[2026-04-12 17:08:27,445] INFO [GroupCoordinator 0]: Stabilized group console-consumer-11359 gene
ration 1 (__consumer_offsets-0) with 1 members (kafka.coordinator.group.GroupCoordinator)
[2026-04-12 17:08:27,454] INFO [GroupCoordinator 0]: Assignment received from leader console-cons
umer-3ecdac69-ff3f-4e4f-ad3b-a2de6612501a for group console-consumer-11359 for generation 1. The
group has 1 members, 0 of which are static. (kafka.coordinator.group.GroupCoordinator)
[2026-04-12 17:17:53,820] INFO [NodeToControllerChannelManager id=0 name=forwarding] Node 0 disco
nnected. (org.apache.kafka.clients.NetworkClient)
```

```
cdn@cdn-Virtual-Machine: ~

cdn@cdn-Virtual-Machine: $ /opt/kafka/bin/kafka-topics.sh --create \
--topic teste \
--bootstrap-server localhost:9092 \
--replication-factor 1 \
--partitions 1 \
Created topic teste.
cdn@cdn-Virtual-Machine: $ /opt/kafka/bin/kafka-topics.sh --list --bootstrap-server localhost:9092
teste
cdn@cdn-Virtual-Machine: $ /opt/kafka/bin/kafka-console-producer.sh \
--topic teste \
--bootstrap-server localhost:9092
>hello kafka
^C
```

```
cdn@cdn-Virtual-Machine: ~

cdn@cdn-Virtual-Machine:~$ /opt/kafka/bin/kafka-console-consumer.sh \
--topic teste \
--from-beginning \
--bootstrap-server localhost:9092
hello kafka
^C
```

Referências de Estudo da Atividade

1. **Documentação oficial do Apache Kafka**
Disponível em: <https://kafka.apache.org/documentation/>
Material principal utilizado para instalação, configuração e entendimento da arquitetura da plataforma de streaming distribuído mantida pela Apache Software Foundation.
2. **Apache Kafka – Quick Start**
Disponível em: <https://kafka.apache.org/quickstart>
Guia prático utilizado como base para criação de tópicos, execução de producers e consumers e validação do ambiente.
3. **Apache Kafka – Operations and Deployment Guide**
Disponível em: <https://kafka.apache.org/documentation/#ops>
Referência utilizada para configuração de brokers, logs, armazenamento e boas práticas de infraestrutura.
4. **Apache ZooKeeper Documentation**
Disponível em: <https://zookeeper.apache.org/doc/current/>
Documentação utilizada para instalação e entendimento do funcionamento do ZooKeeper no modo legado de coordenação de cluster Kafka.
5. **Draw.io (diagrams.net) – Ferramenta de Diagramas**
Disponível em: <https://app.diagrams.net>
Ferramenta utilizada para criação dos fluxogramas, diagramas de arquitetura e BPV do projeto.
6. **Documentação oficial do Hyper-V**
Disponível em: <https://learn.microsoft.com/virtualization/hyper-v-on-windows/>
Referência para criação e configuração de máquinas virtuais no Microsoft Hyper-V.
7. **Guia de criação de máquinas virtuais no Windows**
Disponível em: <https://learn.microsoft.com/windows-server/virtualization/hyper-v/get-started/create-a-virtual-machine-in-hyper-v>
Material complementar para configuração da infraestrutura virtual utilizada na POC.
8. **Documentação oficial Ubuntu**
Disponível em: <https://ubuntu.com/tutorials>
Base utilizada para instalação de pacotes, configuração de rede e preparação do ambiente Linux com Canonical Ubuntu.
9. **Apache Kafka GitHub Repository**
Disponível em: <https://github.com/apache/kafka>
Repositório oficial do Apache Kafka contendo código-fonte, exemplos e scripts utilizados durante os testes.